

Factorisation de Bunch-Parlett et Bunch-Kaufman

MTH8211 - Projet phase 1

Hortense Beaudoin et Téo Dumoutier

Table of Contents

1. Description du projet
2. Théorie
3. Implémentation
4. Tests et performances
5. Discussion
6. Retour sur les objectifs

Description du projet

Si on tente de résoudre un système $Ax = b$ où A est dense, plusieurs solutions existent, mais une majorité sont très coûteuses. La décomposition QR et la SVD existent pour toute matrice A , y compris les matrices indéfinies (i.e il existe x t.q $x^T Ax \leq 0$ et il existe y t.q $y^T Ay \geq 0$), mais elles n'exploitent pas forcément la structure du système à résoudre.

Certaines structures symétriques sont fréquemment rencontrées lors de la résolution de problèmes. On a par exemple vu en classe qu'une autre manière d'exprimer le problème de moindres carrés est de résoudre plutôt le système de point de selle

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}$$

où la matrice formée de A et A^T est symétrique et toujours indéfinie. Sous cette forme, le bloc supérieur gauche est bien évidemment inversible, puisqu'il est égal à l'identité. De manière intéressante, toute autre matrice B symétrique et indéfinie peut être permutée pour adopter une forme où le bloc supérieur gauche est inversible, soit

$$PBP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $D = D^T$ et $E = E^T$ inversible. Ce type de problème est donc particulièrement fréquent et il est intéressant de s'intéresser à une méthode de résolution où la symétrie des matrices est exploitée.

La décomposition de Cholesky n'existe pas pour ce genre de système puisque la matrice considérée n'est pas définie-positive, ce qui est une exigence associée à ce type de factorisation. Une factorisation LDL^* pourrait exister, mais pas systématiquement et elle court le risque d'être numériquement instable. Higham illustre bien ce risque dans son livre avec l'exemple suivant:

$$A = \begin{bmatrix} \epsilon & 1 \\ 1 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 0 \\ 0 & \epsilon - 1/\epsilon \end{bmatrix} \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}$$

A est symétrique et si $\epsilon < 1$, indéfinie. On remarque que dans le cas où $\epsilon = 0$ la factorisation n'existe tout simplement et que le reste du temps, les facteurs $1/\epsilon$ risquent de grossir de manière disproportionnée, menant à un système mal conditionné ou à des erreurs numériques importantes.

Ce projet va donc plutôt s'intéresser aux factorisations de Bunch-Parlett et Bunch-Kaufman, qui exploitent toutes deux l'idée de permutation mentionnée précédemment. La mise en place algébrique étant la même dans les deux cas, nous allons dès maintenant la présenter brièvement.

Théorie

Factorisation en bloc pour matrices symétriques

Tel que mentionné ci-dessus, toute matrice $A \in \mathbb{C}^{n \times n}$ symétrique non-nulle peut être permutée de la manière suivante:

$$PAP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $E \in \mathbb{C}^{s \times s}$, $C \in \mathbb{C}^{n-s \times s}$ et $D \in \mathbb{C}^{(n-s) \times (n-s)}$ avec $s \in 1, 2$ et E inversible. Sous cette forme, il est possible de procéder à la factorisation suivante:

$$PAP^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & D - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}$$

On reconnaît $D - CE^{-1}C^T$ comme étant le complément de Schur de E dans la matrice PAP^T . Ce complément est aussi symétrique, par symétrie des blocs E et D ;

$$(D - CE^{-1}C^T)^T = D^T - CE^{-T}C^T = D - CE^{-1}C^T$$

On peut donc répéter le processus sur cette matrice de dimension $(n-s) \times (n-s)$, puis de nouveau et ainsi de suite jusqu'à l'obtention de la factorisation complète.

À titre d'illustration, imaginons être à l'itération k avec la forme suivante;

$$P_k A_k P_k^T = \begin{bmatrix} E_k & C_k^T \\ C_k & D_k \end{bmatrix} = \begin{bmatrix} I_{s_k} & 0 \\ C_k E_k^{-1} & I_{n-s_k} \end{bmatrix} \begin{bmatrix} E_k & 0 \\ 0 & D_k - C_k E_k^{-1} C_k^T \end{bmatrix} \begin{bmatrix} I_{s_k} & E_k^{-1} C_k^T \\ 0 & I_{n-s_k} \end{bmatrix}$$

On pose alors $A_{k+1} = D_k - C_k E_k^{-1} C_k^T$ et on procède de la même manière.

$$P_{k+1} A_{k+1} P_{k+1}^T = \begin{bmatrix} E_{k+1} & C_{k+1}^T \\ C_{k+1} & D_{k+1} \end{bmatrix} = \begin{bmatrix} I_{s_{k+1}} & 0 \\ C_{k+1} E_{k+1}^{-1} & I_{n-(s_k+s_{k+1})} \end{bmatrix} \begin{bmatrix} E_{k+1} & 0 \\ 0 & D_{k+1} - C_{k+1} E_{k+1}^{-1} C_{k+1}^T \end{bmatrix} \begin{bmatrix} I_{s_{k+1}} & E_{k+1}^{-1} C_{k+1}^T \\ 0 & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

Et ainsi de suite. Les matrices P , L et B sont construites de la façon suivante à chaque itération:

$$P_{k+1} = P_{k+1} P_k \dots P_1$$

$$L_{k+1} = \begin{bmatrix} I_{s_k} & 0 & 0 \\ (C_k E_k^{-1})[1 : s_{k+1}, 1 : s_{k+1}] & I_{s_{k+1}} & 0 \\ (C_k E_k^{-1})[s_{k+1} + 1 : n, 1 : s_{k+1}] & (C_{k+1} E_{k+1}^{-1}) & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

$$B_{k+1} = \begin{bmatrix} E_k & 0 & 0 \\ 0 & E_{k+1} & 0 \\ 0 & 0 & A_{k+2} \end{bmatrix}$$

La différence entre les processus de Bunch-Parlett et Bunch-Kaufman repose dans le choix du pivot, soit de la permutation P_k . Le tout sera exploré plus en détail dans les prochaines sections.

Bunch-Parlett

La stratégie de Bunch-Parlett pour le choix du pivot est aussi appelée *pivotage complet*. Elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonal. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\mu_0 = \max_{i,j} |a_{ij}| =: |a_{pq}|$, ($q \geq p$)

Poser $\mu_1 = \max_i |a_{ii}| =: |a_{rr}|$

Si $\mu_1 \geq \alpha\mu_0$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{pp} & a_{qp} \\ a_{qp} & a_{qq} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et p sont permutées, suivies des colonnes et rangées 2 et q)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} \ddots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \vdots & a_{rr} & & & & & \\ \vdots & & \ddots & & & & \\ \vdots & & & a_{pp} & \cdots & a_{qp} & \\ \vdots & & & \vdots & \ddots & \vdots & \\ \vdots & & & a_{qp} & \cdots & a_{qq} & \\ \vdots & & & & & & \ddots \end{pmatrix}$$

Il est intéressant de regarder de plus près la valeur de α et la manière dont cette dernière est obtenue. Elle est dérivée directement de la borne sur les éléments des blocs de B , elle-même issue des valeurs maximales contenues dans A . Comme cette valeur est reprise dans les processus de Bunch-Kaufman ainsi que dans l'alternative du *rook pivoting*, nous trouvons pertinent d'en présenter la brève dérivation ici. À noter que la dérivation s'effectue de manière semblable pour les deux autres processus et mène au même résultat.

Dérivation de α

Dans le cas du pivot 1×1 , ce dernier est choisi si $\mu_1 \geq \alpha\mu_0$, i.e $\max_i |a_{ii}| \geq \alpha \max_{ij} |a_{ij}|$. Toutes les mises à jour de A via le complément de Schur sont ainsi bornée;

$$|a_{ij}| = |b_{ij} - c_{i1} \frac{1}{e_{11}} c_{1j}| \leq \mu_0 + \frac{\mu_0^2}{\mu_1} \leq \left(1 + \frac{1}{\alpha}\right) \mu_0$$

On peut borner les valeurs de $|a_{ij}|$ de manière semblable pour le cas du pivot 2×2 . On note cette fois le complément de Schur comme

$$a_{ij} = b_{ij} - \begin{bmatrix} c_{i1} & c_{i2} \end{bmatrix} E^{-1} \begin{bmatrix} c_{11} \\ c_{12} \end{bmatrix}$$

où

$$E^{-1} = \frac{1}{\det(E)} \begin{bmatrix} e_{22} & -e_{12} \\ -e_{21} & e_{11} \end{bmatrix}$$

Par symétrie de E

$$\det(E) = e_{11}e_{22} - e_{21}^2 \leq \mu_1^2 - \mu_0^2 \leq (\alpha^2 - 1)\mu_0^2$$

Il est alors posé que $\alpha \in (0, 1)$, de telle sorte que $|\det(E)| \geq (1 - \alpha^2)\mu_0^2$. Il est alors possible de borner supérieurement E^{-1} dans le complément de Schur. En effet

$$|E^{-1}| \leq \frac{1}{(1 - \alpha^2)\mu_0^2} \begin{bmatrix} \alpha & 1 \\ 1 & \alpha \end{bmatrix}$$

et donc

$$|a_{ij}| \leq \mu_0 + \frac{1(1 + \alpha)\mu_0^2}{(1 - \alpha^2)\mu_0} = \left(1 + \frac{2}{1 - \alpha}\right) \mu_0$$

Finalement, pour obtenir la valeur de α , le multiplicateur de μ_0 dans la borne du pivot de 2×2 est rendu équivalent à deux fois le multiplicateur de μ_0 dans la borne du pivot de 1×1 , soit

$$\left(1 + \frac{1}{\alpha}\right)^1 = \left(1 + \frac{2}{1 - \alpha}\right)$$

Ce système peut être résolu et mène à deux valeurs de α , parmi lesquelles on choisit la valeur positive, qui est bien $\alpha = (1 + \sqrt{17})/8$.

Il s'agit d'une dérivation assez simple d'une valeur reprise dans chacune des implémentations auxquelles ce projet s'intéresse. Le concept de choix de pivot et de borne sur l'augmentation des éléments est aussi présent dans d'autres types de factorisation et dans des méthodes de base comme l'élimination de Gauss.

Pseudo-code

Pour l'implémentation, on combine le choix de p à la mise en place de base du problème, vue dans la première section du rapport. Le pseudo-code suivant est ce qui a été suivi pour l'implémentation, présentée dans la partie résultats.

Algorithm 1: Factorisation de Bunch-Parlett (pivotage complet)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k \leq n$  do
4   @viewA[k : n, k : n]
5    $\mu_1 = \max_i |a_{ii}| := |a_{rr}|$ 
6    $\mu_0 = \max_{ij} |a_{ij}| := |a_{pq}|$ 
7   if  $\mu \geq \alpha\mu_0$  then
8      $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
9     Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
10    Permuter les entrées  $k$  et  $r$  dans  $P$ 
11    Insérer le bloc  $E$  dans  $A[k, k]$ 
12    Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14     $k += 1$ 
15    Mettre à jour la vue sur  $A$ 
16  else
17     $a_{pq}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
18    Permuter les lignes et colonnes  $k$  et  $p$  dans  $A$ 
19    Permuter les lignes et colonnes  $k + 1$  et  $q$  dans  $A$ 
20    Permuter les entrées  $k$  et  $p$  dans  $P$ 
21    Permuter les entrées  $k + 1$  et  $q$  dans  $P$ 
22    Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
23    Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
24    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
25     $k += 2$ 
26    Mettre à jour la vue sur  $A$ 
27  end
28 end
```

Bunch-Kaufman

La stratégie de Bunch-Kaufman pour le choix du pivot est aussi appelée *pivotage partiel*. Comme dans la stratégie de Bunch-Parlett, elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonale. La différence repose dans le fait qu'elle évite le scannage complet de la matrice pour faire son choix, le décortiquant plutôt en 3 sections de recherche beaucoup plus petite. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Poser r , l'index de la valeur maximale dans la colonne 1

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{11}| \omega_r \geq \alpha \omega_1^2$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Si $|a_{rr}| \geq \alpha \omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 2 et r sont permutées)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} a_{11} & \cdots & a_{r1}(\omega_1) & \cdots & \cdots & \cdots \\ \vdots & & \vdots & & & \\ a_{r1}(\omega_1) & \cdots & a_{rr} & \cdots & a_{ir}(\omega_r) & \cdots \\ \vdots & & \vdots & & & \\ \vdots & & a_{ir}(\omega_r) & & & \\ \vdots & & \vdots & & & \end{pmatrix}$$

Pseudo-code

Pour l'implémentation, on reprend la majorité de ce qui a été fait pour Bunch-Parlett, mais en ajustant le choix de p . Le pseudo-code suivant est ce qui a été suivi pour l'implémentation.

Algorithm 2: Factorisation de Bunch-Kaufman (pivotage partiel)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4   @viewA[ $k : n, k : n$ ]
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha\omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $r = \text{index}(\max_i |a_{i1}|)$ 
18       $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
19      if  $|a_{11}|\omega_r \geq \alpha\omega_1^2$  then
20         $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
21        Insérer le bloc  $E$  dans  $A[k, k]$ 
22        Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
23        Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
24         $k += 1$ 
25        Mettre à jour la vue sur  $A$ 
26      else
27        if  $|a_{rr}| \geq \alpha\omega_r$  then
28           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
29          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
30          Permuter les entrées  $k$  et  $r$  dans  $P$ 
31          Insérer le bloc  $E$  dans  $A[k, k]$ 
32          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
33          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
34           $k += 1$ 
35          Mettre à jour la vuesur  $A$ 
36        else
37           $a_{r1}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
38          Permuter les lignes et colonnes  $k + 1$  et  $r$  dans  $A$ 
39          Permuter les entrées  $k + 1$  et  $r$  dans  $P$ 
40          Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
41          Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
42          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
43           $k += 2$ 
44          Mettre à jour la vue sur  $A$ 
45        end
46      end
47    end
48  end
49 end
```

Rook pivoting

La stratégie de *Rook pivoting* est utilisée lorsque l'on veut éviter que la norme de L soit trop grande (ce qui peut générer des instabilités dans certains algorithmes.) Plusieurs éléments du processus sont les mêmes que pour Bunch-Kaufman, mais le processus comporte une portion itérative. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

$i = 1$

Répéter jusqu'au choix d'un pivot

Poser r , l'index de la valeur maximale dans la colonne i

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{rr}| \geq \alpha\omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Si $\omega_i = \omega_r$

Utiliser $\begin{bmatrix} a_{ii} & a_{ri} \\ a_{ri} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et i sont permutées, suivies des colonnes et rangées 2 et r)

Sinon

$i = r$, $\omega_i = \omega_r$

Si un pivot 1×1 est trouvé, le pivot est choisi directement. Sinon, le processus cherche un pivot a_{ri} tel que $|a_{ri}|$ est le maximum dans la rangée r et le maximum dans la colonne i .

Pseudo-code

Le pseudo-code est encore une fois semblable à celui des processus précédent, à la différence de la boucle **while** utilisée dans le choix du pivot.

Algorithm 3: Factorisation avec rook-pivoting

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4   @view  $A[k : n, k : n]$ 
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha\omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $j = 1$ 
18      while Un pivot n'a pas été choisi do
19         $r = \text{index}(\max_i |a_{ij}|)$ 
20         $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
21        if  $|a_{rr}| \geq \alpha\omega_r$  then
22           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
23          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
24          Permuter les entrées  $k$  et  $r$  dans  $P$ 
25          Insérer le bloc  $E$  dans  $A[k, k]$ 
26          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
27          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
28           $k += 1$ 
29          Mettre à jour la vuesur  $A$ 
30        else
31          if  $\omega_i = \omega_r$  then
32             $a_{rj}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
33            Permuter les lignes et colonnes 1 et  $j$  dans  $A$ 
34            Permuter les lignes et colonnes 2 et  $r$  dans  $A$ 
35            Permuter les entrées 1 et  $j$  dans  $P$ 
36            Permuter les entrées 2 et  $r$  dans  $P$ 
37            Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
38            Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
39            Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
40             $k += 2$ 
41            Mettre à jour la vue sur  $A$ 
42          else
43             $j = r$ 
44             $\omega_j = \omega_r$ 
45          end
46        end
47      end
48    end
49  end
50 end
```

Implémentation

Stratégies d'optimisation

Différentes stratégies ont été adoptées pour optimiser les performances des fonctions créées, autant d'un point de vue mémoire que vitesse d'exécution.

Tout d'abord, pour se conformer aux normes présentées dans Julia, deux versions de chaque processus a été implantée, soit une version renvoyant les matrices L , B et P au sein d'une **struct** et une autre modifiant la matrice en place A - le nom de cette dernière contient ainsi un **!** pour signifier la modification des éléments d'entrée.

Pour limiter le nombre d'allocations dans la seconde version, les matrices B et L sont stockées directement aux emplacements mémoire de A tout au long de la factorisation. Dans le schéma suivant, les blocs bleus représentent les blocs de la matrice B et les verts, les colonnes de la matrices L . Toutes les valeurs au dessus de la diagonale de bloc sont mises à 0.

$$A = \begin{pmatrix} \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \end{pmatrix} \rightarrow \begin{pmatrix} \square & 0 & 0 & 0 & 0 & 0 & 0 \\ \square & \square & \square & 0 & 0 & 0 & 0 \\ \square & \square & \square & 0 & 0 & 0 & 0 \\ \square & \square & \square & \square & 0 & 0 & 0 \\ \square & \square & \square & \square & \square & \square & 0 \\ \square & \square & \square & \square & \square & \square & 0 \\ \square & \square & \square & \square & \square & \square & \square \end{pmatrix}$$

On peut facilement extraire les matrices B et L de cette forme, puisque la taille des blocs est implicitement indiquée par la présence de valeurs ou de 0 sur la première diagonale supérieure (juste au dessus de la diagonale principale.) Une fonction a été créée pour permettre à l'utilisateur de le faire directement, **extract_L_and_B**.

Dans les deux versions, la symétrie de la matrice est également exploitée pour diminuer le temps d'exécution. En effet, lors de la recherche des valeurs maximales hors diagonales (pour Bunch-Parlett), seul la moitié de la matrice a besoin d'être parcourue. La matrice d'entrée est également établie comme étant Hermitienne, ce qui permet de limiter l'espace mémoire. Les permutations effectuées sont également stockées dans un vecteur P plutôt que dans une matrice de permutation complète.

Les **views** sont utilisées pour limiter le nombre d'allocations et des boucles sont aussi utilisées à plusieurs endroits pour éviter le stockage de vecteurs complets. Pour les pivots de taille 2×2 , le complément de Schur est calculé via son expression algébrique pour encore une fois éviter une allocation de vecteur. Il en va de même pour la mise à jour de L dans le cas 2×2 .

Bunch-Parlett

Les fonctions suivantes implémentent la factorisation de Bunch-Parlett (pivotage complet) selon les deux approches mentionnées précédemment.

```
using LinearAlgebra, SparseArrays
using Test

# Implémentation Bunch-Parlett
function perm_A!(M, r::Int, r::Int)
    """
    Permute les lignes et les colonnes r et r de M hermitienne, où r > r
    """
    if r < r
        error("Index r must be greater than index r ")
    end
end
```

```

n = size(M, 1)

M[r, r], M[r, r] = M[r, r], M[r, r]
M[r, r] = conj(M[r, r])
for k in r+1:r-1
    M[k, r], M[r, k] = conj(M[r, k]), conj(M[k, r])
end
for i in r+1:n
    M[i, r], M[i, r] = M[i, r], M[i, r]
end
for j in 1:r-1
    M[r, j], M[r, j] = M[r, j], M[r, j]
end
end

function bunch_parlett(A::Hermitian)
    """
    Factorise A = LBL' selon la factorisation de Bunch-Parlett, utilisant la technique du pivotage complexe
    Entrée : Matrice A carrée, hermitienne et indéfinie
    Sortie : Matrice L triangulaire inférieure
             Matrice B tridiagonale hermitienne
             Vecteur de permutation P
    """
    A = LowerTriangular(Matrix(A))

    = (1 + sqrt(17))/8
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)
    diagonal = zeros(T, n)
    L = LowerTriangular(Matrix{T}(I, n, n))
    P = collect(1:n)

    k = 1

    while k < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, k:n, k:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage

        , r = findmax(abs.(diag(A_)))

        p, q = k, k
        = zero(Float64)
        for i in k:n
            for j in (i+1):n
                if abs(A[i, j]) >
                    = abs(A[i, j])
                    p, q = i, j
                end
            end
        end
    end

```

```

end

if      *      || k == n # E de taille 1, permute les lignes r et 1

    # Permutation dans A_
    perm_A!(A_, 1, r)

    # Permutation dans L
    for j in 1:k-1
        L[k, j], L[k+r-1, j] = L[k+r-1, j], L[k, j]
    end

    # Permutation dans P
    P[k], P[k+r-1] = P[k+r-1], P[k]

    # Déterminant de E
    inv_det_E = conj(A_[1, 1])/ ^2

    # Complément de Schur
    for i in 2:n_
        for j in 2:i
            A_[i, j] -= inv_det_E*(A_[i, 1] * conj(A_[j, 1]))
        end
    end

    # Calcul de B
    diagonal[k] = A_[1, 1]
    subdiagonal[k] = 0

    # Calcul de L
    L[k+1:n, k] .= inv_det_E*A_[2:n_, 1]

    # Prochain bloc
    k += 1

else      # E de taille 2, P permute 1 et p et 2 et q

    # Permutation dans A_
    perm_A!(A_, 1, p)
    perm_A!(A_, 2, q)

    # Permutation dans L
    for j in 1:k-1
        L[k, j], L[k+p-1, j] = L[k+p-1, j], L[k, j]
    end
    for j in 1:k-1
        L[k+1, j], L[k+q-1, j] = L[k+q-1, j], L[k+1, j]
    end

    # Permutation dans P
    P[k], P[k+p-1] = P[k+p-1], P[k]
    P[k+1], P[k+q-1] = P[k+q-1], P[k+1]

```

```

# Déterminant de E
e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
e_12 = conj(e_21)
det_E = (e_11*e_22 - e_21*e_12)
inv_det_E = conj(det_E)/abs(det_E)^2

# Complément de Schur
for i in 3:n_
    for j in 3:i
        A_[i, j] -= inv_det_E*((A_[i, 1]*e_22 - A_[i, 2]*e_21)*conj(A_[j, 1]) + (A_[i, 2]*e_11 - A_[i, 1]*e_12)*conj(A_[j, 2]))
    end
end

# Calcul de B
subdiagonal[k], subdiagonal[k+1] = A_[2, 1], 0
diagonal[k], diagonal[k + 1] = A_[1, 1], A_[2, 2]

# Calcul de L
for i in k + 2:n
    a_i1, a_i2 = A[i, k], A[i, k+1]
    L[i, k] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
    L[i, k+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
end

# Prochain bloc
k += 2
end

end

if k == n - 1
    subdiagonal[k] = A[k+1, k]
    diagonal[k], diagonal[k+1] = A[k, k], A[k+1, k+1]
else
    diagonal[k] = A[k, k]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))

return L, B, P
end

function bunch_parlett!(A::LowerTriangular)
    """
    Factorise A = LBL' selon la factorisation de Bunch-Parlett, utilisant la technique du pivotage complexe.
    Entrée : Matrice A carrée, hermitienne et indéfinie
    Sortie : Matrice A carrée et hermitienne contenant à la fois L et B :
        - L sur les éléments hors diagonaux
        - B sur les éléments blocs diagonaux ne faisant pas partie de L
    Vecteur de permutation P
    Vecteur de position des blocs 2x2
    """

```

```

    = (1 + sqrt(17))/8
n = size(A, 1)
P = collect(1:n)
B2x2 = Int[]

k = 1
while k < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, k:n, k:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
    , r = findmax(abs.(diag(A_)))

    p, q = k, k
    = zero(Float64)
    for i in k:n
        for j in (i+1):n
            if abs(A[i, j]) >
                = abs(A[i, j])
                p, q = i, j
            end
        end
    end

    if      *    || k == n # E de taille 1, permute les lignes r et 1
        # Permutation dans A_
        perm_A!(A_, 1, r)

        # Permutation dans L
        for j in 1:k-1
            A[k, j], A[k+r-1, j] = A[k+r-1, j], A[k, j]
        end

        # Permutation dans P
        P[k], P[k+r-1] = P[k+r-1], P[k]

        # Déterminant de E
        inv_det_E = conj(A_[1, 1])/ ^2

        # Complément de Schur
        for i in 2:n_
            for j in 2:i
                A_[i, j] -= inv_det_E*(A_[i, 1] * conj(A_[j, 1]))
            end
        end

        # Calcul de L
        A_[2:n_, 1] .*= inv_det_E

        # Prochain bloc
        k += 1

```

```

else

    push!(B2x2, k)

    # Permutation dans A_
    perm_A!(A_, 1, p)
    perm_A!(A_, 2, q)

    # Permutation dans L
    for j in 1:k-1
        A[k, j], A[k+p-1, j] = A[k+p-1, j], A[k, j]
    end
    for j in 1:k-1
        A[k+1, j], A[k+q-1, j] = A[k+q-1, j], A[k+1, j]
    end

    # Permutation dans P
    P[k], P[k+p-1] = P[k+p-1], P[k]
    P[k+1], P[k+q-1] = P[k+q-1], P[k+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    inv_det_E = conj(det_E)/abs(det_E)^2

    # Complément de Schur
    for i in 3:n_
        for j in 3:i
            A_[i, j] -= inv_det_E*((A_[i, 1]*e_22 - A_[i, 2]*e_21)*conj(A_[j, 1]) + (A_[i, 2]*e_11 - A_[i, 1]*e_12)*conj(A_[j, 2]))
        end
    end

    # Calcul de L
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    k += 2
end

end

return P, B2x2
end

function extract_L_and_B(A, B2x2)
    """
    Récupère la matrice L et B à partir de la forme compacte de la factorisation de Bunch-Parlett
    Entrée : Matrice A compacte factorisée selon Bunch-Parlett
    """

```

```

Sortie : Matrice L
         Matrice B
"""
n = size(A, 1)
T = eltype(A)

subdiagonal = zeros(T, n-1)
diagonal = zeros(T, n)
L = LowerTriangular(Matrix{T}(I, n, n))

k = 1
while k < n - 1
    if k in B2x2
        # Matrice B
        subdiagonal[k], subdiagonal[k+1] = A[k+1, k], 0
        diagonal[k], diagonal[k + 1] = A[k, k], A[k+1, k+1]

        # Matrice L
        L[k + 2:n, k] .= A[k + 2:n, k]
        L[k + 2:n, k + 1] .= A[k + 2:n, k + 1]

        # Prochain bloc
        k += 2
    else
        # Matrice B
        subdiagonal[k] = 0
        diagonal[k] = A[k, k]

        # Matrice L
        L[k + 1:n, k] .= A[k + 1:n, k]

        # Prochain bloc
        k += 1
    end
end

if k == n - 1
    subdiagonal[k] = A[k+1, k]
    diagonal[k], diagonal[k+1] = A[k, k], A[k+1, k+1]
else
    diagonal[k] = A[k, k]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))

return L, B
end

```

extract_L_and_B (generic function with 1 method)

Bunch-Kaufman

Les fonctions suivantes implémentent la factorisation de Bunch-Kaufman (pivotage partiel) selon les deux approches mentionnées précédemment.

Rook pivoting

Les fonctions suivantes implémentent la factorisation avec *rook pivoting* selon les deux approches mentionnées précédemment.

Tests et performances

Deux types de tests ont été effectués sur les implémentations. Les premières séries de tests visent à valider que les factorisations fonctionnent sur un ensemble de type de données, soit des `Float64`, des `ComplexF64` et finalement des `BigFloat`. La seconde série de tests vise à évaluer et comparer les performances des différentes implémentations.

Type de données

Les fonctions suivantes servent de manière générale dans les tests qui suivent.

```
function extract_P(vec_P)
    P = Matrix{Int}(I, n, n)
    P = P[vec_P, :]
    return P
end

function perm_line_vec_P(M, vec_P)
    return M[vec_P, :]
end

function perm_column_vec_P(M, vec_P)
    return M[:, vec_P]
end

function transpose_vec_P(vec_P)
    n = size(vec_P, 1)
    vec_P_transposed = zeros{Int, n}
    for i in 1:n
        vec_P_transposed[vec_P[i]] = i
    end
    return vec_P_transposed
end
```

`transpose_vec_P` (generic function with 1 method)

Voici la série de tests sur le processus de Bunch-Parlett.

```
# Fichier de tests pour Bunch-Parlett

N = 25
m, n = 50, 25
for i in 1:N
    A = randn(m, n)
    T = eltype(A)
    K_0 = Hermitian([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
```

```

K = copy(K_0)

L, B, vec_P = bunch_parlett(K)
vec_P_transposed = transpose_vec_P(vec_P)
K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
@test norm(K - K_0) < 1e-10
end

for i in 1:N
A = randn(ComplexF64, m, n)
T = eltype(A)
K_0 = Hermitian([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
K = copy(K_0)

L, B, vec_P = bunch_parlett(K)
vec_P_transposed = transpose_vec_P(vec_P)
K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
@test norm(K - K_0) < 1e-10
@test eltype(L) == T && eltype(B) == T
end

for i in 1:N
A = randn(BigFloat, m, n) + im * randn(BigFloat, m, n)
T = eltype(A)
K_0 = Hermitian([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
K = copy(K_0)

L, B, vec_P = bunch_parlett(K)
vec_P_transposed = transpose_vec_P(vec_P)
K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
@test norm(K - K_0) < 1e-10
@test eltype(L) == T && eltype(B) == T
end

N = 25
m, n = 50, 25
for i in 1:N
A = randn(m, n)
T = eltype(A)
K_0 = LowerTriangular([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
K = copy(K_0)

vec_P, vec_2by2 = bunch_parlett!(K)
L, B = extract_L_and_B(K, vec_2by2)
vec_P_transposed = transpose_vec_P(vec_P)

```

```

K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
K = LowerTriangular(K)
@test norm(K - K_0) < 1e-10
end

for i in 1:N
A = randn(ComplexF64, m, n)
T = eltype(A)
K_0 = LowerTriangular([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
K = copy(K_0)

vec_P, vec_2by2 = bunch_parlett!(K)
L, B = extract_L_and_B(K, vec_2by2)
vec_P_transposed = transpose_vec_P(vec_P)
K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
K = LowerTriangular(K)
@test norm(K - K_0) < 1e-10
@test eltype(K) == T && eltype(L) == T && eltype(B) == T
end

for i in 1:N
A = randn(BigFloat, m, n) + im * randn(BigFloat, m, n)
T = eltype(A)
K_0 = LowerTriangular([sparse(Matrix{T}(I, m, m)) A; A' spzeros(n,n)])
K = copy(K_0)

vec_P, vec_2by2 = bunch_parlett!(K)
L, B = extract_L_and_B(K, vec_2by2)
vec_P_transposed = transpose_vec_P(vec_P)
K = L*B*L'
K = perm_line_vec_P(K, vec_P_transposed)
K = perm_column_vec_P(K, vec_P_transposed)
K = LowerTriangular(K)
@test norm(K - K_0) < 1e-10
@test eltype(K) == T && eltype(L) == T && eltype(B) == T
end

```

Discussion

Comparaison des processus

D'un point de vue théorique, un théorème permet d'assurer la stabilité du processus de Bunch-Kaufman. Ce théorème assure que pour $(A + \Delta A)\hat{x} = b$ la solution du système $\hat{x}$, la perturbation sur la norme de A est bornée supérieurement comme $\|\Delta A\|_M \leq p(n)\rho_n u \|A\|_M + O(u^2)$ où p est quadratique. Ce résultat s'applique si le système est résolu par élimination gaussienne ou en utilisant l'inverse. Aucun résultat semblable n'est associé à Bunch-Parlett.

Pour tous les processus d'intérêt dans ce projet, un autre résultat intéressant est le suivant:

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq p(n)u \operatorname{cond}(A) \operatorname{cond}(|D||L^T|) + O(u^2)$$

$\operatorname{cond}(|D||L^T|)$ n'est pas borné pour le pivotage partiel, mais il l'est pour le *rook pivoting* et cette méthode est donc en théorie moins sensible à un A initial moins bien conditionné.

Comparaison avec l'implémentation Julia

Difficultés rencontrées

Retour sur les objectifs

Les objectifs principaux du projet, pour débiter, sont tels que donné dans l'énoncé;

1. Comprendre et documenter les factorisations de Bunch-Parlett et Bunch-Kaufman (pivotage complet et pivotage partiel.) Fournir des exemples, des schémas explicatifs, faire des liens avec les éléments vus en classe.
 - Voir la section Description du projet et la section Implémentation. La description des factorisations précède le pseudo-code des deux fonctions ainsi que le code lui-même.
 - Quelques schémas y sont utilisés pour illustrer les stratégies de pivot.
 - Un schéma dans la section Stratégies d'optimisation illustre le stockage de la factorisation de façon compacte.
2. Implémenter ces deux factorisation ainsi que l'alternative du *rook pivoting*. Les comparer et identifier leurs avantages et leurs inconvénients respectifs.
 - Pour le pivotage complet, on s'attend à au plus $n^3/6$ comparaisons pour le choix de p . (to check)
 - Pour le pivotage partiel, on s'attend à $O(n^2)$ comparaisons pour le choix de p . (to check)
 - Pour le *rook pivoting*, on s'attend à entre $O(n^2)$ et $O(n^3)$ comparaisons pour le choix de p (to check)
 - Les implémentations dans Julia nous donnerons le référent pour le nombre de flops des implémentations complètes. (to check)
 - L'implémentation de chaque processus est décrite dans la section Implémentation.
 - La sous-section Stratégies d'optimisation explique comment les fonctions ont été optimisées pour réduire le temps d'exécution ainsi que le nombre d'allocation.
 - La section Comparaison des processus compare les 3 processus entre eux ainsi que d'un point de vue théorique.
3. Comparer nos implémentations à celles déjà disponibles dans Julia et tenter d'identifier les sources possibles des différences observées.
 - Julia possède une fonction de base dans LinearAlgebra appelée `BunchKaufman`. La fonction retourne trois matrices, D , U et p . (to check)
 - Nous n'avons pour l'instant pas trouvé d'implémentation de Bunch-Parlett en Julia (pas dans les fonctions de base du moins.) (to check)
4. Joindre la factorisation à des méthodes de résolution de systèmes linéaires.
5. Appliquer l'implémentation à un ensemble de problèmes modèles ainsi qu'à un problème plus spécifique au choix.
 - Nous savons que Julia possède des Packages dédiés à l'optimisation et des 'lots' de problèmes d'optimisation de base, qui peuvent être utiles pour faire du Benchmarking et comparer différentes approches. Nous verrons si ces éléments peuvent servir dans notre cas ou pas.

Sur le temps total du projet ($\sim 20h$), 15% du temps est prévu être alloué à l'étape 1, 50% à l'étape 2 et 3 qui peuvent être faites en parallèle, puisque diverses stratégies sont implémentées, 20% à l'étape 4 et 15% à l'étape 5. En effet, une portion de temps significative doit être allouée à la revue de littérature afin d'acquérir une compréhension solide sur laquelle construire le projet. De plus, il est attendu que la validation et l'optimisation de performance, qui sont des aspects importants des étapes 2 et 3, en plus de l'implémentation en tant que telle, représente un défi majeur afin d'obtenir des implémentations optimisées. Une grande portion du temps doit donc être allouée à ces étapes. Les recherches pour intégrer ces factorisations dans les différentes méthodes de résolution de système linéaire sera probablement le deuxième défi le plus important, suivi par la création de problèmes de tests robustes et adaptés. Ainsi, un peu plus du temps restant est alloué à l'étape 4 qu'à l'étape 5.

Références

[1] : Nicholas J. Higham. Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027. URL <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>

[Lien vers le dépôt Github du projet](#)